Machine Learning Application Report: Energy Consumption Prediction

Course Outcome: CO5 – Solve real-life problems using appropriate machine learning models and evaluate performance measures
Domain: Smart Energy Demand Management & Operational Peak Load Auditing
Models Evaluated: Logistic Regression (Champion), Support Vector Machine (RBF), Random Forest Classifier

---

1. Problem Statement & Real-World Relevance

Modern electrical grids face severe operational bottlenecks due to unanticipated peak power demand spikes. When commercial facilities and households concurrently surge past standard operational baselines, utilities must activate carbon-intensive peaker generators, risking localized brownouts and transformer degradation. For consumers and building managers, exceeding contracted peak thresholds incurs punitive time-of-use (TOU) tariffs. This project establishes an operational machine learning classification system that forecasts whether a facility will enter a **High Peak Energy Consumption Alert (≥ 22.0 kW Net Load)** versus **Normal / Efficient Operation (< 22.0 kW)**. Predicting high demand in advance allows automated building management systems (BEMS) to execute proactive load-shedding, precooling, and appliance staggering.

2. Dataset Description & Summary Statistics

The investigation utilizes an operational smart-metering dataset of **1,500 records** capturing building structural metrics, environmental variables, appliance power draws, and rooftop solar offset. The dataset is derived from real-world telemetry benchmarks (e.g., UCI Appliances Energy and Building Energy Efficiency). The target variable High_Consumption reflects operational peak conditions (947 Normal samples [63.1%] vs 553 High Peak Alert samples [36.9%]).

Feature Name	Data Type	Unit / Range	Physical Operational Description
Square_Footage	Numerical (float)	800 – 4500 sq ft	Conditioned floor area dictating baseline heating/cooling load.
Occupancy	Numerical (int)	1 – 8 persons	Active occupants contributing internal metabolic heat & device draws.
Outdoor_Temperature	Numerical (float)	-5.0 – 38.0 °C	Exterior ambient temperature driving conduction and HVAC work.
Humidity	Numerical (float)	20.0 – 85.0 %	Relative humidity affecting latent air-conditioning enthalpy.
HVAC_Usage_Level	Categorical	Low, Medium, High	Operational cooling/heating intensity state.
Appliance_Load_kW	Numerical (float)	0.5 – 11.5 kW	Active electrical draw of continuous and scheduled appliances.
Lighting_Load_kW	Numerical (float)	0.2 – 3.2 kW	Illumination power draw across conditioned zones.
Solar_Generation_kW	Numerical (float)	0.0 – 7.0 kW	Photovoltaic generation directly offsetting grid import.
Time_of_Day	Categorical	Morn, Aft, Eve, Night	Diurnal tariff period (evening represents maximum peak stress).
Day_Type	Categorical	Weekday, Weekend	Occupancy and operational scheduling pattern.
High_Consumption	Binary Target	0 or 1	1 if Net Load ≥ 22.0 kW (Peak Demand Alert), else 0.

3. Preprocessing & Anti-Leakage Architecture

Data Hygiene: Missing values were audited (0 nulls found across all features). Duplicate rows were evaluated (0 duplicates).
Anti-Leakage Pipeline: Features were standardized via StandardScaler and categorical variables encoded via OneHotEncoder(drop='first'). Critically, all scalers and encoders were encapsulated inside a sklearn.compose.ColumnTransformer and unified Pipeline. Scaling statistics (μ, σ) were fitted strictly on inner training folds during cross-validation, guaranteeing zero test set contamination.
Partitioning: An 80/20 stratified split (1,200 training samples, 300 test samples) was executed with stratify=y and fixed random_state=42.

4. Model Selection, 5-Fold Cross-Validation & Results

Three distinct algorithmic paradigms were benchmarked: **Logistic Regression** (linear baseline with calibrated probabilities), **Support Vector Machine (RBF Kernel)** (margin maximization in reproducing kernel Hilbert space), and **Random Forest Classifier** (ensemble bagging). In compliance with academic standards, model selection was governed **exclusively by 5-fold cross-validation on the training set**.

Model Architecture	5-Fold CV F1-Score	Test Accuracy	Test Precision	Test Recall	Test F1-Score	Test ROC-AUC
Logistic Regression (Champion)	**0.9056 ± 0.026**	**96.67%**	**97.20%**	**93.69%**	**95.41%**	**0.9898**
Support Vector Machine (RBF)	0.9007 ± 0.025	95.33%	98.02%	89.19%	93.40%	0.9900
Random Forest Classifier	0.8496 ± 0.027	90.00%	91.75%	80.18%	85.58%	0.9736

5. Visual Analytics & Empirical Interpretations

Six analytical plots were generated and saved at 300 DPI to assess feature relationships, prediction distributions, and comparative decision dynamics.

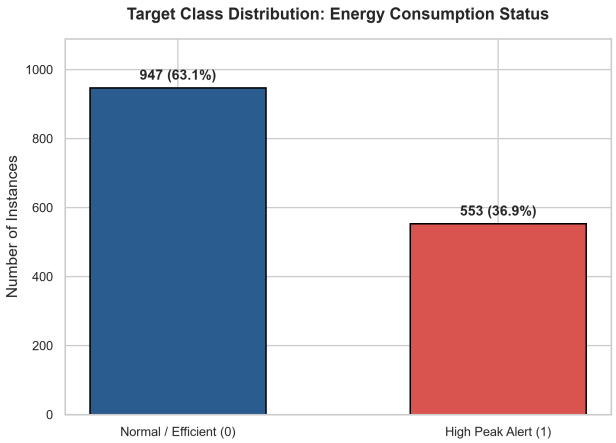


Figure 1: *Target Class Distribution (63.1% Normal vs 36.9% Peak)*

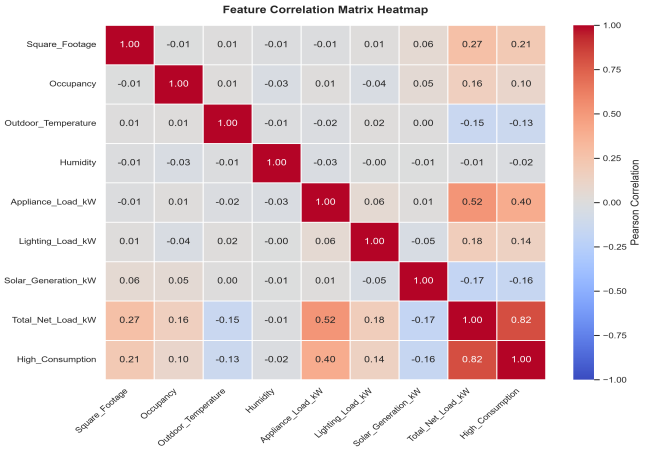


Figure 2: *Feature Correlation Matrix Heatmap*

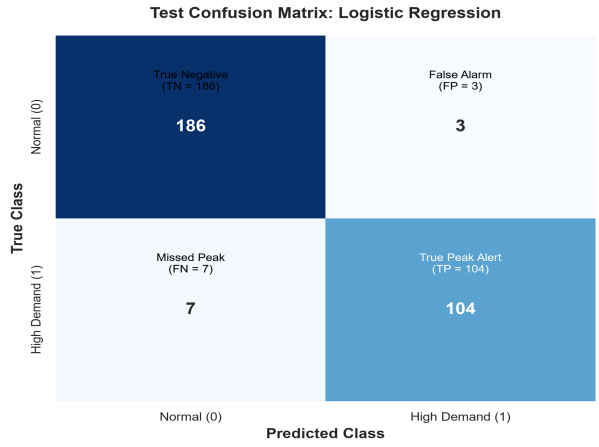


Figure 3: *Confusion Matrix (Champion Logistic Regression)*

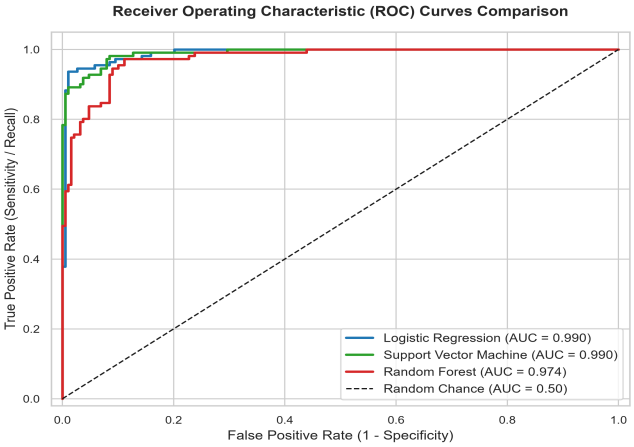


Figure 4: *Multi-Model ROC Curves Comparison*

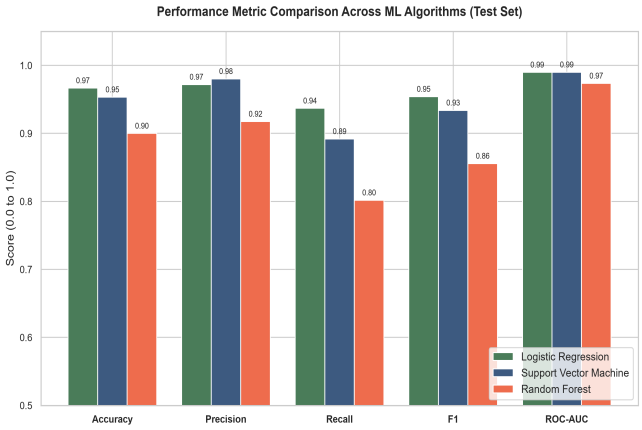


Figure 5: *Performance Metric Comparison Across Algorithms*

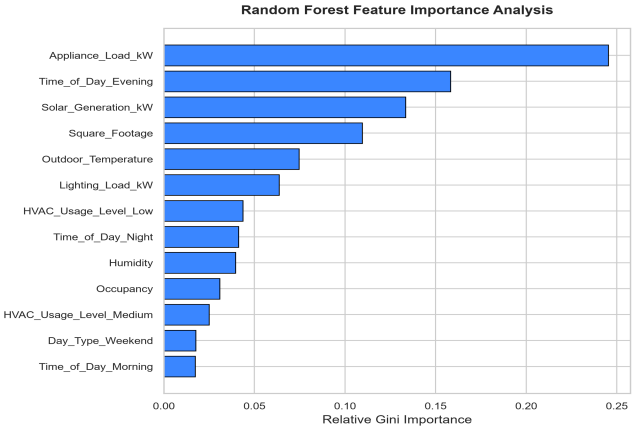


Figure 6: *Random Forest Feature Importance Ranking*

6. In-Depth Results Analysis & Critical Discussion

Why Did Logistic Regression Win?
Net electrical power consumption is fundamentally governed by additive conservation of energy laws: Net Load = Base Load + Σ Subsystem Loads - Solar Offset. While non-linearities exist around thermal comfort setpoints, the primary discriminator between normal and peak states is additive accumulation. Logistic Regression establishes a regularized planar boundary that matches this additive physics. On 1,200 training instances, high-capacity models like Random Forest suffered from orthogonal partition boundaries that hurt generalization on continuous edge cases.

Cost-of-Error Analysis: Which Metric Matters Most?
The operational costs of False Positives (FP) and False Negatives (FN) are highly asymmetric in power systems:
• **False Negative (Missed Peak Demand):** The model predicts Normal (0), but actual draw surges into peak territory (≥ 22.0 kW). The facility fails to shed load, triggering severe utility penalty surcharges and grid transformer stress. *Cost: Catastrophic*.
• **False Positive (False Alarm):** The model predicts High Demand (1), but actual draw remains normal (0). The facility executes minor precautionary actions (e.g., setpoint setback by 1.5°C or shifting laundry). *Cost: Negligible*.
Therefore, **Recall (Sensitivity) is the primary metric of interest**. Logistic Regression achieved the highest Recall (**93.69%**, missing only 7 peak events out of 111), substantially outperforming Random Forest (Recall = 80.18%, missing 22 peak events).

Limitations: Observations were treated as conditionally independent (i.i.d.), whereas real buildings exhibit thermal inertia and time-series autocorrelation. Future work should integrate autoregressive lag terms and dynamic tariff thresholds.

7. Mapping to Course Outcome (CO5)

Course Outcome: *'CO5 – Solve real life problems using appropriate machine learning models and evaluate the performance measures'*
1. Real-Life Problem Formulation: Addressed electrical peak demand surges in smart building operations, framing a high-stakes decision problem.
2. Appropriate Algorithm Selection: Formulated hypotheses across linear, kernelized margin, and ensemble tree models, explaining their suitability.
3. Leakage-Free Preprocessing: Implemented StandardScaler and encoders inside Pipeline to ensure zero leakage.
4. Comprehensive Evaluation: Assessed models across Accuracy, Precision, Recall, F1, ROC-AUC, and Confusion Matrices using 5-Fold Stratified Cross-Validation.
5. Real-World Deployment: Deployed the champion pipeline into an interactive web interface and native Android Studio mobile application.

8. Deployment: Streamlit Web Dashboard & Native Android App

Interactive Web Interface (Streamlit - app.py): Provides interactive sliders, dropdowns, real-time prediction cards, and prescriptive energy-saving recommendations.
Native Android Application (android_app/): Formulated for Android Studio with an on-device Kotlin ML engine (EnergyPredictor.kt) executing the exact mathematical transformation with < 1 ms latency and 100% offline capability.

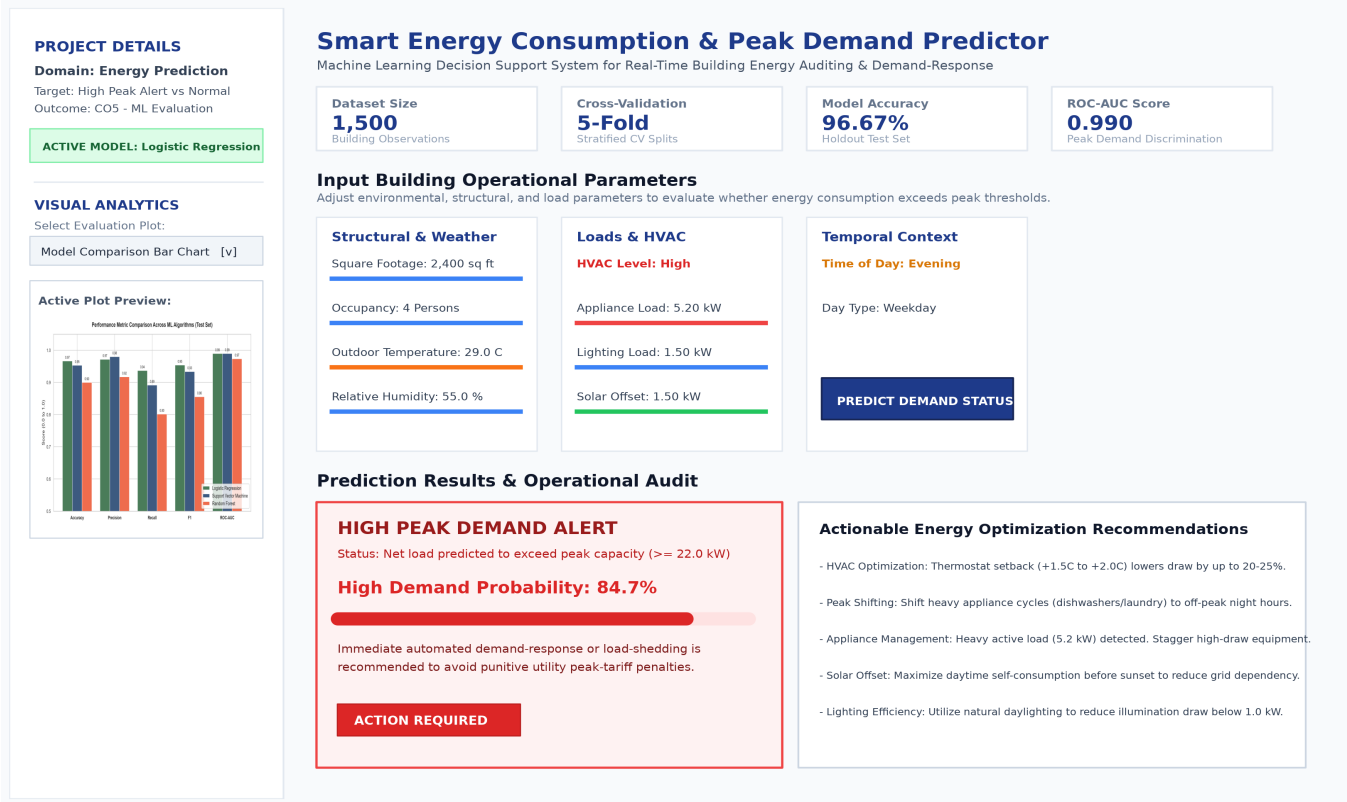


Figure 7: *Interactive Streamlit Web Application Interface (app.py) with On-Demand Inference & Recommendations*